

Chatwoot + Outline Bot

Integration Blueprint

Template-Based Response (Tanpa LLM)

Zero Cost Solution

Tech Stack Options: Python | Go | JavaScript

Technical Blueprint Document

June 2026

Tujuan

Membuat middleware bot yang otomatis menjawab chat masuk di Chatwoot berdasarkan dokumentasi yang ada di Outline, menggunakan template-based response tanpa LLM.

Ketika ada chat masuk, sistem akan membaca pesan tersebut, mencari dokumen yang relevan di Outline, lalu membalas dengan jawaban yang diformat menggunakan template yang sudah disiapkan.

Komponen Sistem

Komponen	Teknologi	Fungsi
Middleware Bot	Python / Go / JavaScript (pilih salah satu)	Service utama
Chatwoot	Self-hosted (sudah ada)	Platform chat
Outline	Self-hosted (sudah ada)	Sumber dokumentasi
Docker	Docker + Docker Compose	Deployment

Total dependency eksternal baru: ZERO - hanya menambah 1 service kecil (middleware bot).

Perbandingan Bahasa Pemrograman

Middleware bot ini bisa dikembangkan menggunakan 3 pilihan bahasa. Pilih berdasarkan keahlian dan kebutuhan tim.

Bahasa	Framework	Performa	Kemudahan	Deploy
Python	FastAPI / Flask	Sedang	Sangat Mudah	Docker / VPS
Go	Gin / Echo	Sangat Tinggi	Sedang	Single Binary
JavaScript	Express.js / Fastify	Tinggi	Mudah	Docker / VPS

Option 1: Python

Python | FastAPI / Flask

KELEBIHAN (+)

- + Sintaks paling mudah dibaca dan dipahami

KEKURANGAN (-)

- Sedikit lebih lambat dibanding Go dan Node.js untuk high concurrency

Contoh Struktur Project (Python / FastAPI)

```
chatwoot-outline-bot/
|-- main.py                # Entry point, FastAPI app
|-- routes/
|   +-- webhook.py         # POST /webhook handler
|-- services/
|   |-- outline_service.py # Search docs di Outline
|   |-- chatwoot_service.py # Send reply ke Chatwoot
|   +-- template_service.py # Format response
|-- templates/
|   +-- responses.py       # Koleksi template
|-- config.py              # Environment variables
|-- requirements.txt
|-- Dockerfile
+-- docker-compose.yml
```

Dependencies Utama

```
fastapi      # Web framework
uvicorn      # ASGI server
httpx       # Async HTTP client
python-dotenv # Env variable loader
pydantic     # Data validation
```

Option 2: Go (Golang)

Go (Golang) | Gin / Echo

KELEBIHAN (+)

- + Performa paling tinggi - cocok untuk volume chat yang besar

KEKURANGAN (-)

- Learning curve lebih tinggi, terutama untuk developer yang belum kenal Go

Contoh Struktur Project (Go / Gin)

```
chatwoot-outline-bot/
|-- main.go           # Entry point
|-- handlers/
|   +-- webhook.go    # POST /webhook handler
|-- services/
|   |-- outline.go    # Search docs di Outline
|   |-- chatwoot.go   # Send reply ke Chatwoot
|   +-- template.go   # Format response
|-- models/
|   +-- types.go      # Struct definitions
|-- config/
|   +-- config.go     # Environment variables
|-- go.mod
|-- go.sum
|-- Dockerfile
+-- docker-compose.yml
```

Dependencies Utama

```
github.com/gin-gonic/gin    # Web framework
github.com/joho/godotenv    # Env variable loader
net/http (stdlib)          # HTTP client (built-in)
```

Option 3: JavaScript (Node.js)

JavaScript (Node.js) | Express.js / Fastify

KELEBIHAN (+)

- + Paling banyak contoh dan tutorial di internet

KEKURANGAN (-)

- Tidak ada type safety (jika pakai JS murni, bukan TypeScript)

Contoh Struktur Project (JavaScript / Express)

```
chatwoot-outline-bot/
|-- src/
|   |-- index.js           # Entry point, Express server
|   |-- routes/
|   |   +-- webhook.js      # POST /webhook handler
|   |-- services/
|   |   |-- outline.service.js # Search docs di Outline
|   |   |-- chatwoot.service.js # Send reply ke Chatwoot
|   |   +-- template.service.js # Format response
|   |-- templates/
|   |   +-- responses.js     # Koleksi template
|   +-- config/
|       +-- index.js         # Environment variables
|-- package.json
|-- Dockerfile
+-- docker-compose.yml
```

Dependencies Utama

```
express    # Web framework
axios      # HTTP client
dotenv     # Env variable loader
nodemon    # Auto-restart (dev only)
```

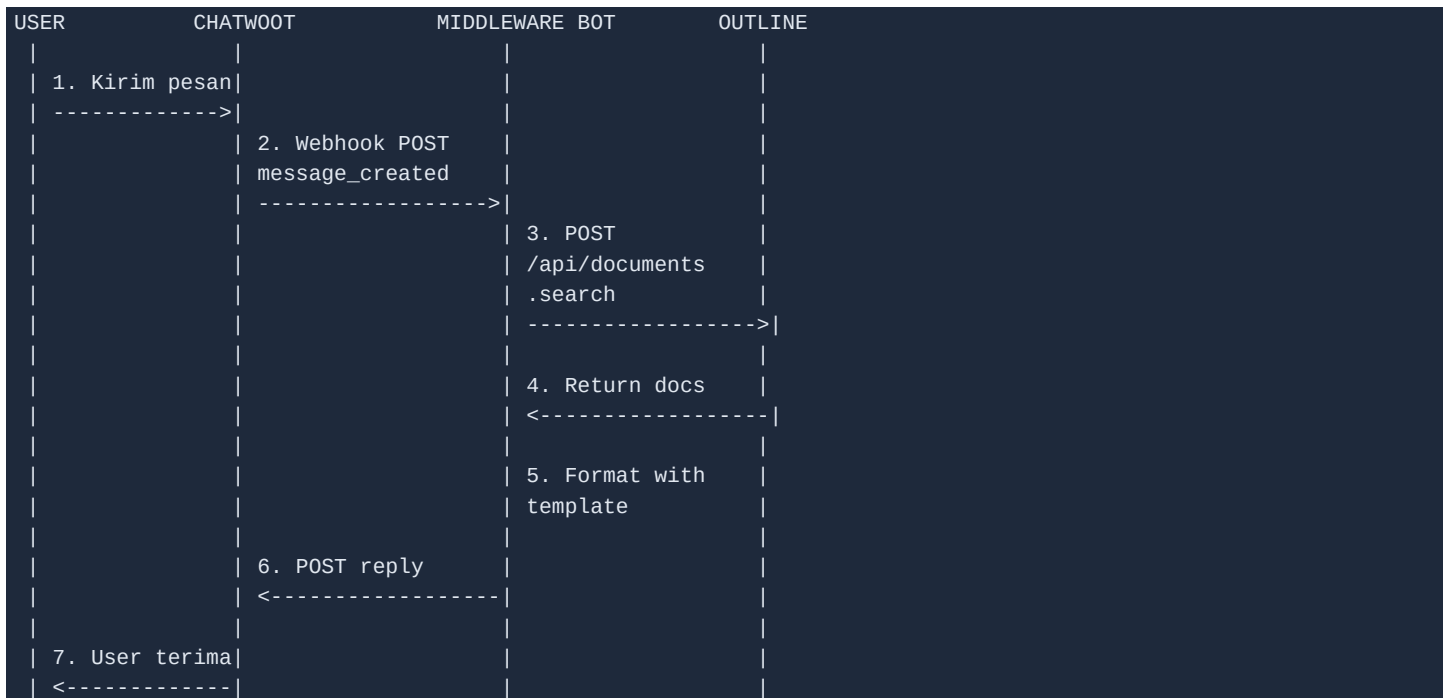
Rekomendasi Pemilihan Bahasa

Kondisi	Rekomendasi
Tim familiar dengan Python	Python + FastAPI
Tim familiar dengan JS/web dev	JavaScript + Express.js
Volume chat sangat tinggi (1000+/hari)	Go + Gin
Deploy di shared hosting	PHP (alternatif lain)
Mau yang paling simpel & cepat jalan	JavaScript + Express.js
Butuh auto-docs API	Python + FastAPI
Minimal resource server	Go + Gin

Untuk kebanyakan use case dengan tim kecil dan volume chat normal, JavaScript (Express.js) atau Python (FastAPI) adalah pilihan terbaik karena paling mudah dikembangkan dan di-maintain.

System Flow

Flow Utama



Decision Logic

```

Chat masuk (webhook)
|
v
Apakah message dari bot sendiri? --> Ya --> IGNORE (cegah loop)
|
No
v
Apakah message_type = "incoming"? --> Bukan --> IGNORE
|
Ya
v
Extract isi pesan
|
v
Search ke Outline API (/api/documents.search)
|
v
Hasil ditemukan?
|-- Ya --> Ambil top 1-3 dokumen paling relevan
|         --> Format dengan template
|         --> Kirim reply ke Chatwoot
|
+-- Tidak --> Kirim response fallback
|              --> (Optional) Auto-assign ke human agent

```


API yang Digunakan

Chatwoot API

Endpoint	Method	Fungsi
Webhook (config di Chatwoot)	POST (incoming)	Menerima event chat masuk
/api/v1/accounts/{id}/conversations/{id}/messages	POST	Mengirim reply
/api/v1/accounts/{id}/conversations/{id}/assignments	POST	Assign ke human agent

Outline API

Endpoint	Method	Fungsi
/api/documents.search	POST	Mencari dokumen berdasarkan query

Template Response

Jawaban Ditemukan

```
Halo! Saya menemukan informasi yang mungkin membantu:

{document_title}

{document_snippet}

[Baca selengkapnya]({document_url})

---
Apakah ini menjawab pertanyaan kamu?
Jika belum, ketik "agent" untuk terhubung ke tim support.
```

Jawaban Tidak Ditemukan

```
Halo!

Mohon maaf, saya belum menemukan informasi yang sesuai
dengan pertanyaan kamu di dokumentasi kami.

Saya akan menghubungkan kamu ke tim support kami.
Mohon tunggu sebentar ya.
```

Requirements / Prerequisites

Yang Harus Sudah Ada

Item	Detail
Chatwoot running	Self-hosted, sudah aktif
Outline running	Self-hosted, sudah ada dokumentasi
Server/VPS	Minimal 512MB RAM (bisa di server yang sama)
Docker & Docker Compose	Terinstall di server

Yang Perlu Disiapkan

Item	Cara Mendapatkan
Chatwoot API Access Token	Settings > Account > Access Token
Outline API Token	Settings > API > Create Token
Chatwoot Webhook URL	Settings > Integrations > Webhooks
Outline Base URL	URL Outline (misal: docs.company.com)
Chatwoot Base URL	URL Chatwoot (misal: chat.company.com)
Chatwoot Account ID	Terlihat di URL dashboard Chatwoot
Bot Agent di Chatwoot	Buat agent khusus untuk bot

Total Biaya

Item	Biaya
Middleware Bot (server)	\$0 (jalankan di server existing)
LLM	\$0 (tidak pakai)
Chatwoot	\$0 (self-hosted)
Outline	\$0 (self-hosted)
TOTAL	\$0/bulan

Estimasi Development

Phase	Detail	Waktu
Setup project + config	Init project, env, Docker setup	1 hari
Core development	Webhook + Outline search + Chatwoot reply	2-3 hari
Template & formatting	Design & implement response templates	1 hari
Testing & edge cases	End-to-end testing, error handling	1-2 hari
Deployment + fine-tuning	Deploy, monitor, adjust templates	1 hari
TOTAL		5-7 hari kerja

Keamanan

Aspek	Implementasi
Webhook validation	Verifikasi source webhook dari Chatwoot
API token	Disimpan di environment variable, tidak hardcode
Rate limiting	Batasi request ke Outline agar tidak overload
Loop prevention	Cek sender ID, ignore jika dari bot sendiri

Future Enhancement (Optional)

Enhancement	Deskripsi
Caching	Cache hasil search yang sering ditanya (Redis/in-memory)
Analytics	Log pertanyaan tidak terjawab untuk improve docs
Multi-language	Deteksi bahasa user, respond sesuai bahasa
Feedback loop	User bisa rate jawaban bot (helpful/not helpful)
LLM upgrade	Tambah LLM layer jika butuh response lebih natural